Unit 1: Functional Dependency and Normalization

A **functional dependency (FD)** is a relationship between two sets of attributes in a table. We write X -> Y when the value of X uniquely determines the value of Y. For example, in a Student table, StudentID -> StudentName, because every student ID belongs to exactly one student name.

Normalization is the process of organising the columns and tables of a relational database to reduce data redundancy and to avoid insertion, update and deletion anomalies. A badly designed table stores the same fact many times. If that fact changes, every copy must be updated, otherwise the data becomes inconsistent.

Normalization works by decomposing a large table into smaller, well-structured tables and linking them with foreign keys. The decomposition must be lossless, which means the original data can be rebuilt by joining the tables.

The main normal forms are First Normal Form (1NF), Second Normal Form (2NF), Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF). Each form adds a stricter rule than the one before it.

Unit 1 (continued): Normal Forms

First Normal Form (1NF): A table is in 1NF if every column holds atomic (single, indivisible) values and there are no repeating groups. A column that stores a list such as 'Maths, Physics' violates 1NF.

Second Normal Form (2NF): A table is in 2NF if it is in 1NF and has no **partial dependency**. A partial dependency exists when a non-key attribute depends on only a part of a composite primary key. Example: in Enrollment(StudentID, CourseID, StudentName, Grade) the key is (StudentID, CourseID), but StudentName depends only on StudentID. To reach 2NF we move StudentName to a separate Student table.

Third Normal Form (3NF): A table is in 3NF if it is in 2NF and has no **transitive dependency**. A transitive dependency exists when a non-key attribute depends on another non-key attribute. Example: in Employee(EmpID, DeptID, DeptName), EmpID -> DeptID and DeptID -> DeptName, so DeptName depends on EmpID only through DeptID. To reach 3NF we create a Department(DeptID, DeptName) table.

Difference between 2NF and 3NF: 2NF removes partial dependencies (dependence on part of a composite key), while 3NF removes transitive dependencies (dependence of one non-key attribute on another non-key attribute).

BCNF: A table is in BCNF if for every functional dependency X -> Y, X is a super key. BCNF is a slightly stronger version of 3NF.

Unit 2: Transactions

A **transaction** is a sequence of database operations that is treated as a single logical unit of work. A classic example is transferring money: subtract from account A and add to account B. Both steps must succeed, or neither should happen.

Transactions follow the **ACID properties**: **Atomicity** (all operations happen or none happen), **Consistency** (the database moves from one valid state to another valid state), **Isolation** (concurrent transactions do not interfere with each other) and **Durability** (once a transaction commits, its changes are permanent even after a crash).

A transaction passes through these states: Active, Partially Committed, Committed, Failed and Aborted. If a transaction fails, the system rolls back its changes.

Serializability means that the result of running transactions concurrently is the same as running them one after another in some order. It is the standard correctness rule for concurrent schedules.

Unit 3: Graph Search - BFS and DFS

Breadth-First Search (BFS) explores a graph level by level. It starts at a source node, visits all its neighbours, then visits the neighbours of those neighbours. BFS uses a **queue** (first in, first out). In an unweighted graph BFS finds the shortest path in terms of number of edges.

Depth-First Search (DFS) explores a graph by going as deep as possible along one branch before backtracking. DFS uses a **stack** (last in, first out) or recursion. It uses less memory than BFS on wide graphs but does not guarantee the shortest path.

Difference between BFS and DFS: BFS uses a queue and explores level by level, guaranteeing the shortest path in unweighted graphs. DFS uses a stack or recursion and explores depth first, so it may find a longer path first. Both have time complexity O(V + E), where V is the number of vertices and E is the number of edges.